

JAVA 17 OCP · YDS STUDY SERIES

Unit 03 · Making Decisions — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge altı adet **OCF tarzi özgün çalışma sorusu** içerir. Sorular gerçek sınavdan alınmamıştır. Akış sorularında önce compile-time scope'u, sonra hangi branch'in çalışacağını, en son output'u belirle.

Sorular

Soru 1

Odak: Switch expression

Aşağıdaki programın çıktısı nedir?

```
public class SwitchValue {
    public static void main(String[] args) {
        int number = 2;
        String result = switch (number) {
            case 1 -> "A";
            case 2 -> {
                yield "B";
            }
            default -> "C";
        };
        System.out.println(result);
    }
}
```

- A. **A**
- B. **B**
- C. **C**
- D. Kod derlenmez.

Soru 2

Odak: Pattern variable scope

Aşağıdaki kod için hangisi doğrudur?

```
public class PatternScope {
    public static void main(String[] args) {
        Object value = "java";
        if (value instanceof String text || text.length() > 2) {
            System.out.println(text);
        }
    }
}
```

- A. **java** yazdırır.
- B. Hiçbir şey yazdırmaz.
- C. Kod derlenmez.
- D. Runtime'da **ClassCastException** oluşur.

Soru 3**Odak:** Dangling `else`

Braces kullanılmayan nested `if` yapısında bir `else` hangi `if` ile eşleşir?

- A. Aynı indentation seviyesindeki ilk `if` ile
- B. Kendisine en yakın, henüz eşleşmemiş `if` ile
- C. Her zaman dıştaki `if` ile
- D. Compiler indentation'a bakarak karar verir.

Soru 4**Odak:** Switch selector types

Java 17'de aşağıdaki type'lardan **hangi ikisi** doğrudan bir `switch` selector'ının type'ı olabilir?

- A. `byte`
- B. `String`
- C. `long`
- D. `double`
- E. `boolean`

Soru 5**Odak:** `continue` ve traditional `for`

Traditional `for` loop içinde `continue`; çalıştığında sıradaki adım hangisidir?

- A. Loop derhal sona erer.
- B. Update expression çalışır; ardından boolean condition yeniden kontrol edilir.
- C. Update expression atlanır ve loop body baştan başlar.
- D. Method'dan çıkılır.

Soru 6**Odak:** English → Turkish / YDS

Cümleyi doğal Türkçeye çevir ve `whereas` ile karşılaştırılan iki kuralı yaz:

A switch expression must be exhaustive, whereas a switch statement does not have to cover every possible value.

Cevaplar ve açıklamalar

1. B — B

Selector 2 olduğu için ikinci rule seçilir. Block kullanılan rule bir değer üretmek için `yield "B";` kullanır. A ve C yanlış branch'leri seçer. D yanlıştır; bütün olası `int` değerleri `default` ile kapsandığından expression exhaustive'tir.

2. C — Does not compile / derlenmez

`||` operator'ının sağ tarafı, sol taraf `false` olduğunda çalışır. Tam da bu yolda pattern match başarısız olabileceği için `text` sağ operand'da scope içinde değildir. Condition `true` olduğunda bunun pattern match'ten mi yoksa sağ operand'dan mı geldiği garanti edilemediği için `text`, `if` body içinde de scope'ta değildir. A ve B output varsaymadan önce compile kontrolünü atlar. D yanlıştır; explicit cast yoktur ve kod runtime'a ulaşmaz. `||` yerine `&&` kullanılsaydı `text`, sağ operand'da ve `if` body'nin true-path'inde kullanılabilirdi.

3. B

`else`, kendisine en yakın unmatched `if` ile eşleşir. Indentation Java grammar'ını değiştirmez; bu nedenle A, C ve D güvenilir kurallar değildir. Braces kullanmak insan okuyucu için niyeti açık hâle getirir.

4. A ve B

- **A doğru:** `byte` ve wrapper'ı `Byte` desteklenen selector type'larından.
- **B doğru:** `String` Java 17 `switch` yapısında kullanılabilir.
- **C, D ve E yanlış:** `long`, `double` ve `boolean` doğrudan desteklenen selector type'ları değildir.

5. B

Traditional `for` akışı `body` → `update` → `condition` biçiminde devam eder. `continue`, body'nin kalanını atlar ama update expression'ı atlamaz. A, `break` davranışdır; C update adımı yanlış atlar; D ise `return` davranışdır.

6. Örnek çeviri

“Bir `switch` expression exhaustive olmak, yani bütün olası değerleri kapsamak zorundadır; buna karşılık bir `switch` statement'in her olası değeri kapsaması gerekmez.”

`whereas`, expression için zorunlu olan exhaustiveness ile statement için bulunmayan bu zorunluluğu karşılaştırır. `does not have to`, “yapmamalıdır” değil, “yapmak zorunda değildir” anlamına gelir.